

TP08: Simulating MapReduce Jobs (Final Report)

1. Dataset

```
%%writefile weblogs.txt
# Date, Time, IP, Method, URL, Status, ResponseSize
(see full dataset used in Colab)
```

2. Main Code: Requests per Status

```
with open("weblogs.txt","r") as f:
    lines = f.readlines()

def mapper(lines):
    mapped=[]
    for line in lines:
        if line.startswith("#"):
            continue
        parts=line.strip().split(",")
        status=parts[5]
        mapped.append((status,1))
    return mapped

def group(mapped):
    grouped={}
    for key,value in mapped:
        if key not in grouped:
            grouped[key]=[]
        grouped[key].append(value)
    return grouped

def reducer(grouped):
    result={}
    for key in grouped:
        result[key]=sum(grouped[key])
    return result

mapped=mapper(lines)
grouped=group(mapped)
result=reducer(grouped)
```

Result:

```
HTTP 200: 10
HTTP 404: 5
HTTP 500: 3
HTTP 403: 2
```

Explanation: counts number of requests per status code.

3. Bonus 1: Requests per URL

```
def mapper_url(lines):
    mapped=[]
    for line in lines:
        if line.startswith("#"):
            continue
        parts=line.strip().split(",")
        url=parts[4]
        mapped.append((url,1))
    return mapped

mapped=mapper_url(lines)
grouped=group(mapped)
result=reducer(grouped)
```

Result:

```
/index.html: 6
/products.html: 3
/contact.html: 3
/about.html: 2
/images/logo.png: 2
/checkout: 3
/login: 2
```

Explanation: counts number of visits per URL.

4. Bonus 2: Total Size per Status

```
def mapper_size(lines):
    mapped=[]
    for line in lines:
        if line.startswith("#"):
            continue
        parts=line.strip().split(",")
        status=parts[5]
        size=int(parts[6])
        mapped.append((status,size))
    return mapped

mapped=mapper_size(lines)
grouped=group(mapped)
result=reducer(grouped)
```

Result:

```
HTTP 200: 8154 bytes
HTTP 404: 2560 bytes
HTTP 500: 384 bytes
HTTP 403: 128 bytes
```

Explanation: sums response size for each status.

5. Bonus 3: Errors Only

```
def mapper_errors(lines):
    mapped=[]
    for line in lines:
        if line.startswith("#"):
            continue
        parts=line.strip().split(",")
        status=parts[5]
        if status!="200":
            mapped.append((status,1))
    return mapped

mapped=mapper_errors(lines)
grouped=group(mapped)
result=reducer(grouped)
```

Result:

```
HTTP 404: 5
HTTP 500: 3
HTTP 403: 2
```

Explanation: filters and counts only error requests.